

NeuroLog AI: setup and user guide

A practical guide to the repaired local application

1. What this project does

NeuroLog collects application log messages, displays their severity, groups similar messages, and helps you investigate unusual events. The repaired version runs locally without MongoDB, Firebase, or an AI-provider account. It uses SQLite in `data/neurolog.db`, React on port 5173, and Flask on port 5001.

The real ML pipeline is TF-IDF plus DBSCAN. This repository does not include an XGBoost model, HDBSCAN, a validated 94% accuracy result, or automatic remediation. The assistant offers advice; it never executes commands. Local demo responses are deterministic and are explicitly labeled. Set a Groq API key to obtain live LLM responses.

Original project: <https://github.com/Sai-Nitin123/Neurolog-AI>. The original MIT license and attribution are retained.

2. Start on Windows

Install Node.js 22.12 or newer and Python 3.12. Open PowerShell in this project's root folder. The current workspace already has dependencies installed.

Terminal 1:

```
.\.venv\Scripts\python.exe LogIntel_engine\api.py
```

Terminal 2:

```
cd logintel_ui
npm run dev
```

Open `http://127.0.0.1:5173`. Both terminals must remain running. Press Ctrl+C in each terminal to stop the services. Data persists between restarts.

For a fresh checkout, run this from the root:

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-lock.txt
cd logintel_ui
npm ci
cd ..
```

Alternatively, `powershell -ExecutionPolicy Bypass -File .\start.ps1` installs dependencies, starts the backend, and keeps the frontend in the terminal. It stops its backend when the frontend exits. Pass `-Python` with the path to Python if Python is not on PATH. Do not start this launcher while the same ports are already in use.

3. Your first five minutes

1. Click **Load sample logs**. Nine records are added: six successful requests and three unusual messages. Each click adds another batch.
2. Open **Live Monitor**. View severity distribution and the recent-log health score. The score is a heuristic: $\max(0, 100 - 12 * \text{critical_log_count})$ over the latest 100 logs; it is not a measured uptime percentage.
3. Click **Run clustering**. At least five records are needed. The initial sample batch produces three outliers.
4. Open **Vector Analysis**. Inspect isolated messages flagged by DBSCAN. An outlier means unusual text, not proof of a security incident.
5. Open **Log Explorer**. Search for database. Choose **Errors / critical** to filter syslog levels 0 through 3.
6. Open **AI Assistant**, ask what to investigate, or click **Analyze & Solve** beside a log. Without Groq configuration, the local advisor returns a labeled generic investigation checklist.
7. Open **Configuration** to create ingestion keys, change appearance, save retention, or purge logs. Purging asks for confirmation and permanently deletes this workspace's logs.

The incident-marker card reflects explicit warning, recovery, or failure markers reported in log messages. It does not predict a crash or verify a remote service's current state. Charts and tables refresh by polling every two to five seconds.

4. Connect your own application

In Configuration, enter an application name and click **Create Key**. Copy the key immediately. Only its hash and final six characters are stored, so the full key cannot be recovered after leaving the page. Keep it in a private environment variable, never in frontend code or a Git commit.

PowerShell example:

```
$env:NEUROLOG_INGEST_KEY = 'PASTE_YOUR_NEW_KEY'
$payload = @{
    source = 'my-api'
    message = 'Database connection refused'
    severity_level = 2
} | ConvertTo-Json
Invoke-RestMethod -Method Post `
    -Uri 'http://127.0.0.1:5001/api/ingest' `
    -Headers @{'x-api-key'=$env:NEUROLOG_INGEST_KEY} `
    -ContentType 'application/json' -Body $payload
```

The API assigns the timestamp, workspace owner, and canonical severity label. Syslog levels are: 0 emergency, 1 alert, 2 critical, 3 error, 4 warning, 5 notice, 6 info, 7 debug. Use the source field to distinguish applications in the same workspace. All keys in a workspace feed that same workspace; they are not separate user accounts.

The optional simulator uses the same key: set NEUROLOG_INGEST_KEY, install dependencies in server with `npm ci`, then run `npm run simulate`. Stop it with Ctrl+C. The optional Node gateway (`npm start` inside server) forwards `/api/log` to Flask; it is not required for the normal app.

5. How the parts work together

Application -> POST /api/ingest + API key -> Flask validation -> SQLite -> React polling.

Run clustering -> read latest 100 messages -> TF-IDF -> DBSCAN -> save `cluster_id` and `ml_anomaly` -> Vector Analysis.

Assistant question -> recent log context -> Groq, if configured -> Markdown answer. Without Groq in demo mode, a deterministic local advisor answers instead.

TF-IDF represents each message by the relative importance of its words. DBSCAN groups messages that are close in that vector space (`eps=0.5`, `min_samples=2`). Messages outside a group receive cluster -1. Clustering runs on demand; it is not a continuously trained failure-prediction model. Repeated warnings may form a normal-looking text cluster, and harmless unique messages can be outliers, so inspect severity and context together.

`LogIntel_engine/api.py` owns the supported API. `logintel_ui/src/pages` contains the screens. `src/api.js` adds Firebase tokens and handles API failures. `data/neurolog.db` stores records, key hashes, and retention settings. `server` is an optional compatibility gateway and simulator. `log_courier.py` and `te.py` are legacy MongoDB/file-processing examples, separate from the supported dashboard pipeline.

6. Configure live services

For live Groq responses, copy `.env.example` to `.env` in the root, set `GROQ_API_KEY` and an available `GROQ_MODEL`, then restart Flask. AI responses send recent log text to Groq. Use only logs you intend to share with that provider and redact sensitive fields first. Requests time out and return a clear provider error if the service is unavailable.

For real accounts, create your own Firebase project. Enable Email/Password and, optionally, Google sign-in. Add your frontend hostname to Firebase's authorized domains. Copy `logintel_ui/.env.example` to `logintel_ui/.env.local`, set `VITE_DEMO_MODE=false`, and fill all four Firebase values. In root `.env`, set `NEUROLOG_DEMO=false` and `GOOGLE_APPLICATION_CREDENTIALS` to a private Firebase Admin service-account JSON path. Restart both services. The backend verifies Firebase ID tokens and derives ownership from them; it ignores supplied user IDs.

Demo mode is one local workspace and bypasses sign-in. Keep it bound to loopback. A public deployment requires demo mode off, TLS, a production WSGI server, deliberate CORS origins, rate limiting, backups, and operational monitoring. SQLite is suitable for a small local installation, not a high-throughput distributed log platform. Retention deletes old logs when logs are read; it is not a scheduled cleanup job. Slack, email delivery, and alternate assistant styles are explicitly disabled because they are not implemented.

7. Troubleshooting and checks

- Backend unavailable: start Flask on port 5001; the frontend proxy routes /api there.
- Port already in use: stop the earlier process before restarting. The frontend uses a strict port so it will not silently move.
- No outliers: add at least five meaningful messages and run clustering. Repeated or stop-word-only messages may produce no usable vocabulary.

- Invalid ingestion key: create a new key in Configuration and update the sender's environment variable.
- Firebase sign-in fails: check your configuration, enabled providers, authorized domains, and matching frontend/backend mode.
- AI provider error: check Groq credentials, model availability, and network access.
- Reset only the logs: use Purge Workspace. It retains ingestion keys and settings. To reset all local data, stop Flask and remove `data/neurolog.db` after backing it up.

Run backend checks from the root with `.\.venv\Scripts\python.exe -m pytest -q`. In `logintel_ui`, run `npm run lint`, `npm run build`, and `npm audit`. In server, run `npm run check` and `npm audit`. GitHub Actions repeats these checks on pushes and pull requests.

Verified locally: API regression tests, frontend lint and production build, both npm dependency audits, and browser walkthrough of the local demo. Live Firebase and Groq integration require your own credentials and have not been exercised with a real account.